PROJECT REPORT
ON

FSM-Based Elevator Controller System

By

P Daphne Christina Ruby

Nutheti Pavithra

Kesihambigai

Digital Systems Design

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

TABLE OF CONTENTS

Sl. No.	Topics	Page No
1.	**INTRODUCTION**	1
1.1	Literature Review	1
1.2	Aim	1
1.3	Objectives	1
2.	**HARDWARE COMPONENTS USED**	**2**
3.	**WORK DONE**	**3**
3.1	Design of Finite state machine(FSM)	3
3.2	Implementation of Elevator controller module	3
3.3	Integration with top module	4
3.4	Simulation and verification in ModelSim	5
4.	**SOFTWARE CODING (Verilog HDL)**	**6**
4.1	Elevator contoller code	6
4.2	Testbench	9
4.3	Elevator top code	12
5.	**RESULTS & DISCUSSION**	**14**
6.	**FUTURE WORK**	**18**
7.	**REFERENCES**	**19**

1.INTRODUCTION

1.1.Literature Survey

Elevator control systems are classic examples of finite state machines (FSMs) used in real-time embedded systems. The system must respond to various user inputs—such as floor requests, door operations, and safety signals—while maintaining safe, efficient, and deterministic movement between floors. Implementing this control logic on Field Programmable Gate Arrays (FPGAs) offers a flexible, fast, and parallel hardware-based solution compared to traditional microcontroller or PLC implementations.

This project shows the design and FPGA implementation of an Elevator Controller System using a Finite State Machine (FSM) approach. The system demonstrates how an elevator's operations—such as floor movement, direction control, and door opening—can be effectively managed using digital logic on an FPGA board.

1.2.Aim

The main aim of this project is to design, simulate, and implement an elevator controller system using a Finite State Machine (FSM) on FPGA hardware. The controller manages elevator movement between floors, door operation, and user requests efficiently with real-time response and deterministic control.

1.3. Objectives

- To model elevator operation using FSM architecture for states such as *Idle,Move*, and *Door Open*.
- To implement the FSM in Verilog HDL and simulate it to verify correct functionality.
- To synthesize the design on an FPGA board (e.g., Altera/Intel or Xilinx) for hardware demonstration.
- To demonstrate real-time control of elevator operations (Up, Down, Door) using FPGA LEDs and switches.

To compare the advantages of hardware (FPGA) implementation over software-based controllers in terms of speed, reliability, and concurrency.

2. HARDWARE COMPONENTS USED

Sl. No.	Component	Purpose / Function	Key Features
1	**FPGA Development Board** (e.g., *Intel/Altera DE1, Cyclone IV* or *Xilinx Spartan-6*)	Main platform used to implement the elevator controller FSM and interface inputs/outputs.	• Reconfigurable logic fabric• On-board 50 MHz clock• GPIO pins for switches, LEDs, and buttons• Supports Verilog/VHDL synthesis
2	**Clock Source (50 MHz Crystal)**	Provides the base clock signal for FPGA logic operations and timing generation.	• Stable and accurate frequency• Divided internally to obtain slow human-visible operation (~1 Hz)
3	**Slide Switches (SW[1:0])**	Used as input to select the desired floor number in the elevator system.	• Easy manual control• Two-bit binary input for four possible floors
4	**Push Buttons (KEY[1:0])**	KEY[0] = Reset input; KEY[1] = Request input for floor call.	• Debounced mechanical switches• Active-low logic• Used for reset and call request control
5	**LED Indicators (LEDR[2:0])**	Display the elevator's operation status (Up, Down, Door Open).	• LED0 → Up movement• LED1 → Down movement• LED2 → Door open indication
6	**Power Supply (5 V DC Adapter)**	Provides operating voltage to the FPGA board and connected peripherals.	• Stable 5 V regulated output• Over-voltage and current protection
7	**USB-Blaster / JTAG Cable**	Used to program the FPGA with synthesized Verilog bitstream.	• High-speed configuration• Compatible with Quartus/Vivado tools
8	**Computer System (Host PC)**	Used for code development, simulation, synthesis, and FPGA programming.	• Runs Quartus Prime / Vivado Design Suite• Provides UART/JTAG communication
9	**Breadboard and Jumper Wires (if external I/O used)**	For connecting external LEDs, switches, or sensors during testing.	• Easy circuit modification• Reliable prototyping connections

3. WORK DONE

3.1. Design of Finite state machine(FSM)

The first step was to design the elevator control logic using the Finite State Machine (FSM) model.

Key points:

The entire elevator operation was divided into three main states:

> **IDLE**: The elevator waits for a floor request while remaining stationary.
>
> **MOVE**: The elevator moves either up or down depending on the requested floor.
>
> **DOOR**: The elevator door opens for a fixed duration when the destination floor is reached.

Transitions between states occur based on user input and floor comparisons.

The FSM ensures a predictable and safe control flow, avoiding random or undefined states.

This design provides a clear framework for movement, direction, and timing operations.

3.2 .Implementation of Elevator controller module

Once the FSM structure was finalized, the main controller was implemented in Verilog HDL in a module named elevator_controller.

Main steps performed:

Defined all necessary inputs and outputs such as clock, reset, request, request_floor, and control signals (up, down, door_open).

Declared three states — IDLE, MOVE, and DOOR — as parameters.

Wrote an always block triggered on the positive edge of the clock to handle FSM transitions.

Implemented logic for movement direction:

If requested floor > current floor → Move up.

If requested floor < current floor → Move down.

Added a timer counter in the DOOR state to keep the door open for a short time before returning to IDLE.

Included reset logic to initialize all signals to their default state (floor 0, door closed).

This module acts as the core brain of the elevator, managing all decisions and actions based on the FSM.

3.3 Integration with Top Module

A top-level Verilog module named elevator_top was created to interface the controller with the actual FPGA hardware.

Key integration features:

Used the FPGA's 50 MHz clock and implemented a clock divider to generate a slower clock (around 1 Hz) for observable operation.

Mapped slide switches (SW) to represent the requested floor inputs.

Used push buttons (KEY) for reset and floor request signals.

Connected LEDs (LEDR) to display system outputs:

LED0 → Elevator moving Up

LED1 → Elevator moving Down

LED2 → Door Open indicator

Verified proper synchronization between input signals, FSM transitions, and LED outputs.

This integration allowed the design to be demonstrated visually on FPGA hardware.

3.4 Simulation and Verification

Before hardware implementation, the design was tested through simulation using ModelSim/Vivado tools.

Steps followed:

A testbench was created to apply various combinations of floor requests and reset signals.

The waveforms were analyzed to confirm correct state transitions and output behavior.

Checked that:

The elevator moved up/down as expected.

The door opened for the correct time interval.

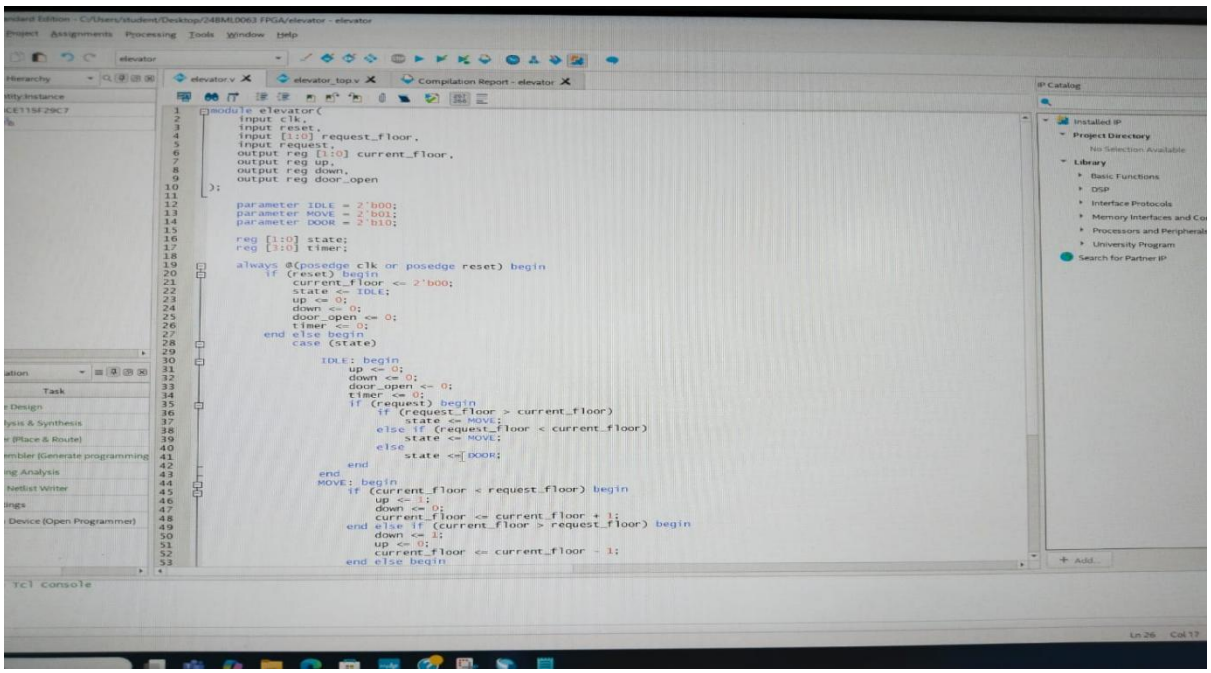
The FSM correctly returned to the IDLE state after each request.

After successful simulation, the design was synthesized and loaded onto the FPGA board using Quartus Prime.

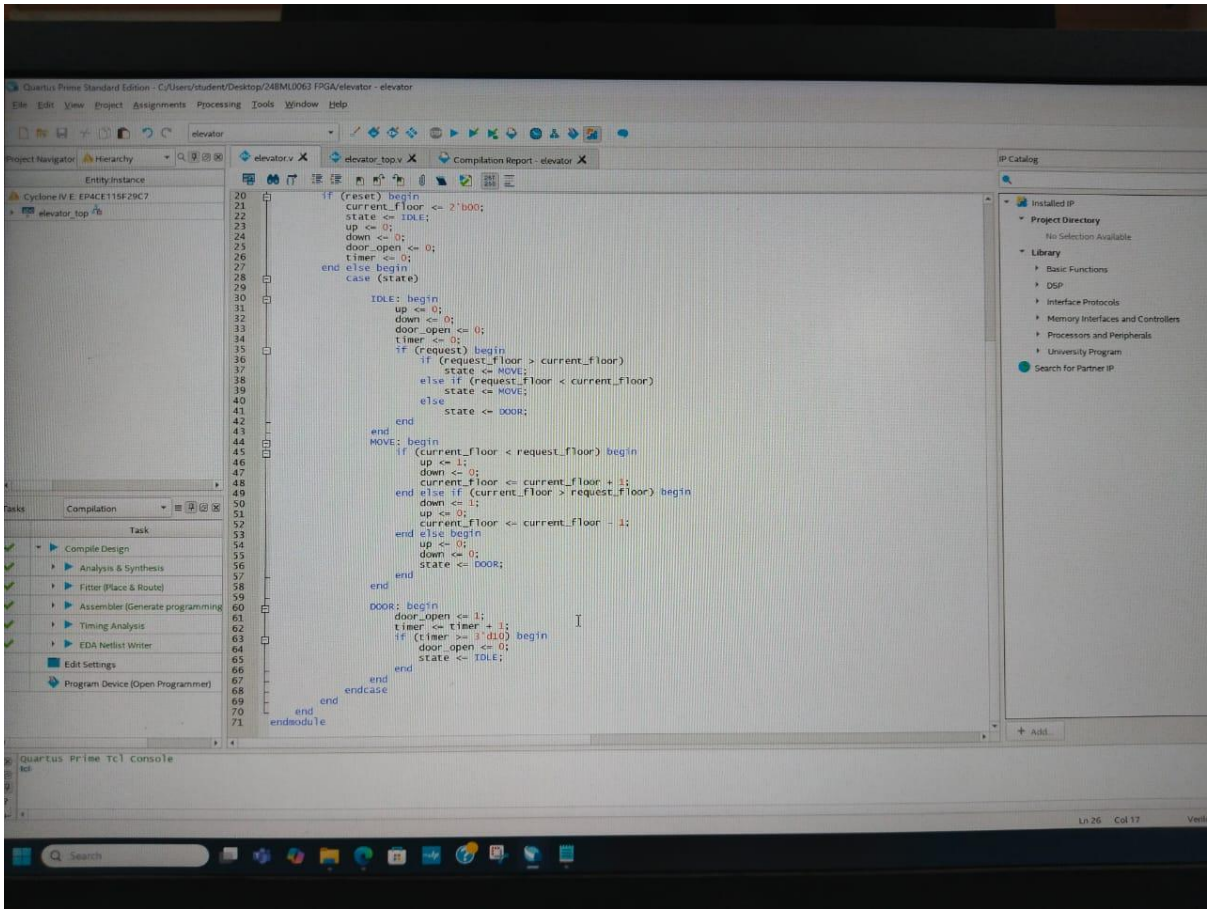
The hardware test confirmed that LEDs responded correctly, and the elevator control logic functioned as intended.

4.SOFTWARE CODING (Verilog HDL)

4.1. Elevator contoller code



```
module elevator(
    input clk,
    input reset,
    input [1:0] request_floor,
    input request,
    output reg [1:0] current_floor,
    output reg up,
    output reg down,
    output reg door_open
);

    parameter IDLE = 2'b00;
    parameter MOVE = 2'b01;
    parameter DOOR = 2'b10;

    reg [1:0] state;
    reg [3:0] timer;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            current_floor <= 2'b00;
            state <= IDLE;
            up <= 0;
            down <= 0;
            door_open <= 0;
            timer <= 0;
        end else begin
            case (state)

                IDLE: begin
                    up <= 0;
                    down <= 0;
                    door_open <= 0;
                    timer <= 0;
                    if (request) begin
                        if (request_floor > current_floor)
                            state <= MOVE;
                        else if (request_floor < current_floor)
                            state <= MOVE;
                        else
                            state <= DOOR;
                    end
                end
                MOVE: begin
                    if (current_floor < request_floor) begin
                        up <= 1;
                        down <= 0;
                        current_floor <= current_floor + 1;
                    end else if (current_floor > request_floor) begin
                        down <= 1;
                        up <= 0;
                        current_floor <= current_floor - 1;
                    end else begin
```

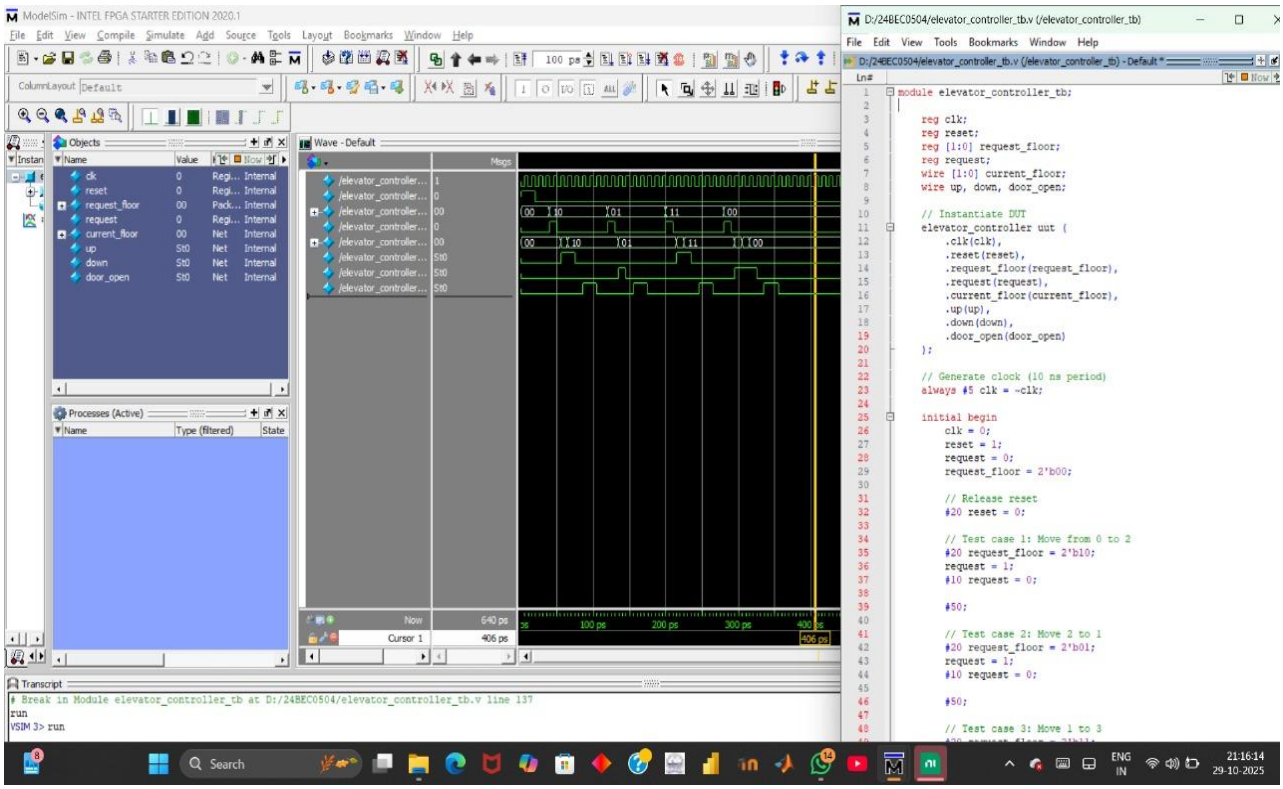


```
        if (reset) begin
            current_floor <= 2'b00;
            state <= IDLE;
            up <= 0;
            down <= 0;
            door_open <= 0;
            timer <= 0;
        end else begin
            case (state)

                IDLE: begin
                    up <= 0;
                    down <= 0;
                    door_open <= 0;
                    timer <= 0;
                    if (request) begin
                        if (request_floor > current_floor)
                            state <= MOVE;
                        else if (request_floor < current_floor)
                            state <= MOVE;
                        else
                            state <= DOOR;
                    end
                end
                MOVE: begin
                    if (current_floor < request_floor) begin
                        up <= 1;
                        down <= 0;
                        current_floor <= current_floor + 1;
                    end else if (current_floor > request_floor) begin
                        down <= 1;
                        up <= 0;
                        current_floor <= current_floor - 1;
                    end else begin
                        up <= 0;
                        down <= 0;
                        state <= DOOR;
                    end
                end

                DOOR: begin
                    door_open <= 1;
                    timer <= timer + 1;
                    if (timer >= 3'd10) begin
                        door_open <= 0;
                        state <= IDLE;
                    end
                end
            endcase
        end
    end
endmodule
```

```
module elevator_controller(
    input clk,
    input reset,
    input [1:0] request_floor,
    input request,
    output reg [1:0] current_floor,
    output reg up,
    output reg down,
    output reg door_open
);

    parameter IDLE = 2'b00;
    parameter MOVE = 2'b01;
    parameter DOOR = 2'b10;

    reg [1:0] state;
    reg [3:0] timer;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            current_floor <= 2'b00;
            state <= IDLE;
            up <= 0;
            down <= 0;
            door_open <= 0;
            timer <= 0;
        end else begin
            case (state)

                IDLE: begin
                    up <= 0;
                    down <= 0;
                    door_open <= 0;
                    timer <= 0;
                    if (request) begin
                        if (request_floor > current_floor)
                            state <= MOVE;
                        else if (request_floor < current_floor)
                            state <= MOVE;
                        else
```

```
                state <= DOOR;
            end
        end

        MOVE: begin
            if (current_floor < request_floor) begin
                up <= 1;
                down <= 0;
                current_floor <= current_floor + 1;
            end else if (current_floor > request_floor) begin
                down <= 1;
                up <= 0;
                current_floor <= current_floor - 1;
            end else begin
                up <= 0;
                down <= 0;
                state <= DOOR;
            end
        end

        DOOR: begin
            door_open <= 1;
            timer <= timer + 1;
            if (timer >= 3'd10) begin
                door_open <= 0;
                state <= IDLE;
            end
        end
    endcase
  end
end
endmodule
```

4.2. Testbench

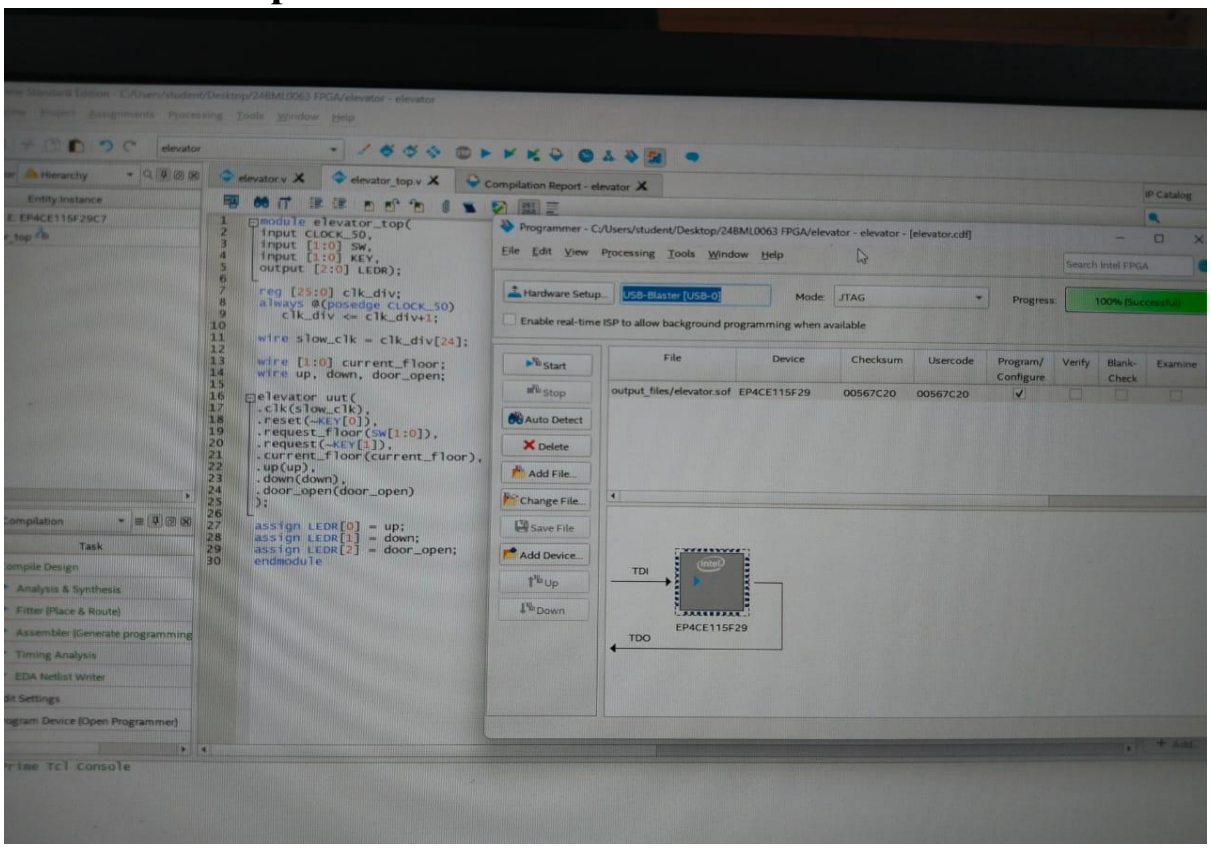



```
module elevator_controller_tb;

    reg clk;
    reg reset;
    reg [1:0] request_floor;
    reg request;
    wire [1:0] current_floor;
    wire up, down, door_open;

    // Instantiate DUT
    elevator_controller uut (
        .clk(clk),
        .reset(reset),
        .request_floor(request_floor),
        .request(request),
        .current_floor(current_floor),
        .up(up),
        .down(down),
        .door_open(door_open)
    );
```

```
// Generate clock (10 ns period)
always #5 clk = ~clk;

initial begin
    clk = 0;
    reset = 1;
    request = 0;
    request_floor = 2'b00;

    // Release reset
    #20 reset = 0;

    // Test case 1: Move from 0 to 2
    #20 request_floor = 2'b10;
    request = 1;
    #10 request = 0;

    #50;

    // Test case 2: Move 2 to 1
    #20 request_floor = 2'b01;
    request = 1;
    #10 request = 0;

    #50;

    // Test case 3: Move 1 to 3
    #20 request_floor = 2'b11;
    request = 1;
    #10 request = 0;

    #50;

    // Test case 4: Move 3 to 0
    #20 request_floor = 2'b00;
    request = 1;
    #10 request = 0;

    #100;
```

```
        // End simulation
        #50;
        $stop; // ModelSim equivalent to stop sim
    end

    // Optional: text output in ModelSim console
    initial begin
        $display("Time\tFloor\tReq_Floor\tUp\tDown\tDoor");
        $monitor("%0t\t%b\t%b\t%b\t%b\t%b",
            $time, current_floor, request_floor, up, down, door_open);
    end

endmodule
```

4.3.Elevator top code




```
module elevator_top(
   input CLOCK_50,       // 50 MHz FPGA clock
   input [1:0] SW,       // Switches to select floor
   input [1:0] KEY,      // KEY[0]=reset, KEY[1]=request
   output [2:0] LEDR     // LEDs for Up, Down, Door
);

   // Clock divider to slow down the 50MHz clock
   reg [25:0] clk_div;
   always @(posedge CLOCK_50)
      clk_div <= clk_div + 1;
   wire slow_clk = clk_div[24];  // ~1 Hz

   Wire [1:0] current floor;
   Wire up,down,door_open;

   // Instantiate elevator controller
   elevator_controller uut (
      .clk(slow_clk),
```

```
        .reset(~KEY[0]),        // Active-low reset
        .request_floor(SW[1:0]),
        .request(~KEY[1]),      // Active-low button
        .current_floor(current_floor),
        .up(up),
        .down(down),
        .door_open(door_open)
    );

    // Output LEDs for movement and door
    assign LEDR[0] = up;        // LED0 = Up
    assign LEDR[1] = down;      // LED1 = Down
    assign LEDR[2] = door_open;  // LED2 = Door open

endmodule
```

5. RESULTS & DISCUSSION

Example 1: Elevator Moving from Floor 0 → 3

Test Setup:

Initial Floor: 0

Requested Floor: 3

Request Input: HIGH (KEY[1] pressed)

Observation:

The elevator starts from floor 0 in the **IDLE** state.

Upon receiving the floor request, the FSM transitions to the **MOVE** state.

The **Up LED (LEDR[0])** glows, indicating upward motion.

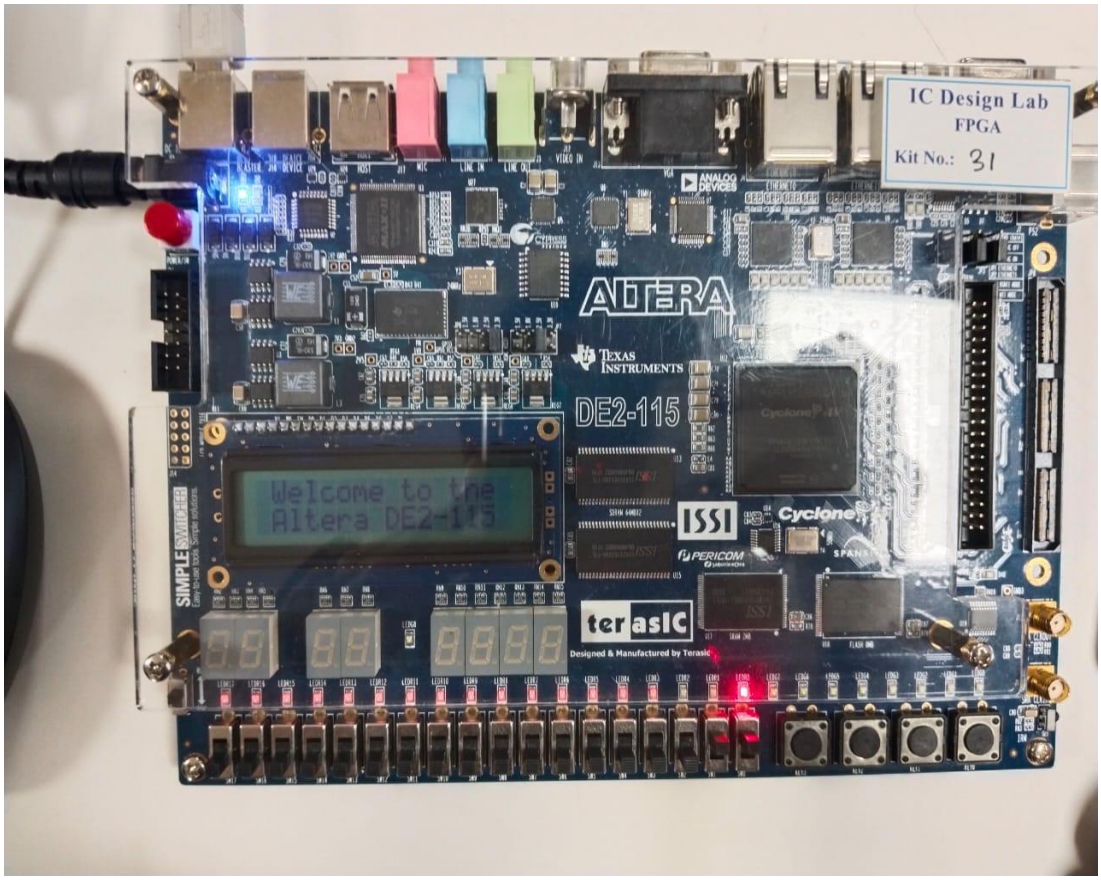
The `current_floor` value increases sequentially:
→ 0 → 1 → 2 → 3 (one floor per clock pulse).

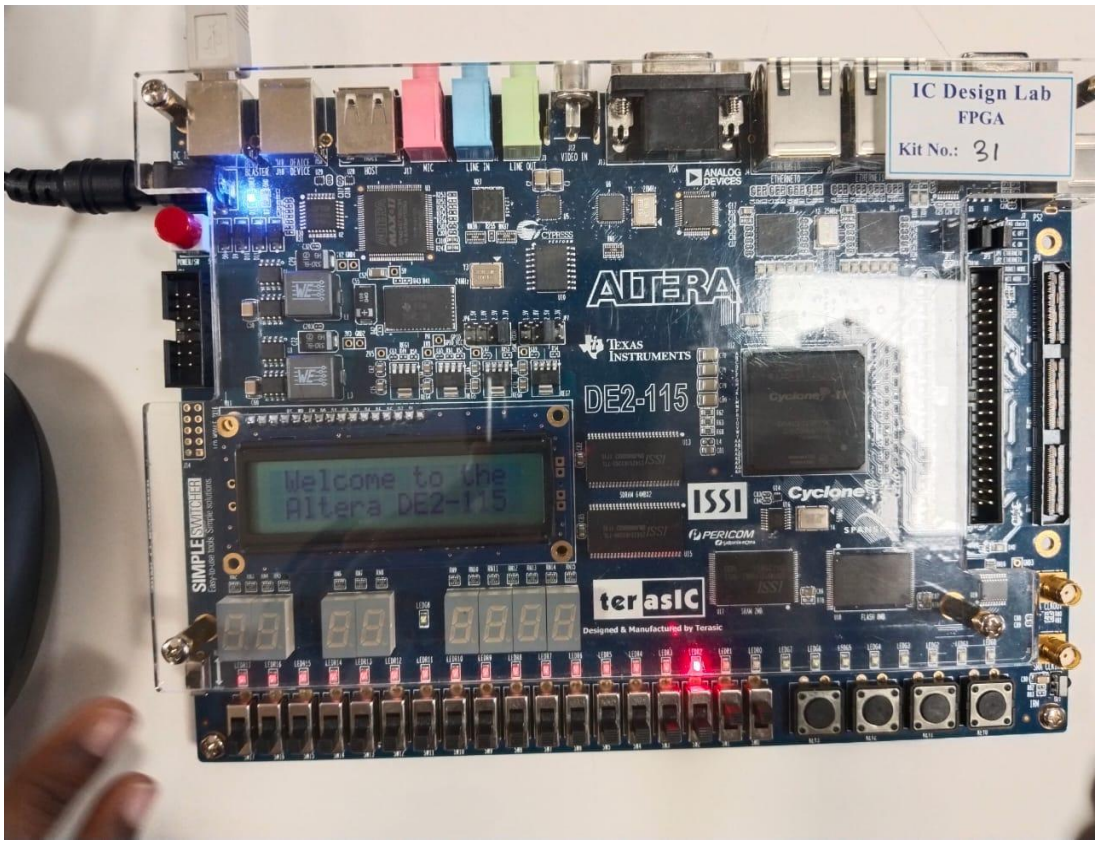
When floor 3 is reached, the **MOVE** state transitions to the **DOOR** state.

The **Door LED (LEDR[2])** turns ON for a fixed duration (10 clock counts), showing door open operation.

After the timer expires, the FSM returns to the **IDLE** state, and all LEDs turn OFF.

Result:
The elevator successfully moved upward through all intermediate floors, stopped accurately at floor 3, opened the door for the programmed delay, and returned to idle operation.


IC Design Lab
FPGA
Kit No.: 31
ALTERA
DE2-115
Welcome to the
Altera DE2-115
ISSI
terasIC


IC Design Lab
FPGA
Kit No.: 31
ALTERA
DE2-115
Welcome to the
Altera DE2-115
ISSI
terasIC

Example 2: Elevator Moving from Floor 3 → 1

Test Setup:

Initial Floor: 3

Requested Floor: 1

Request Input: HIGH (KEY[1] pressed)

Observation:

The elevator begins at floor 3 and receives a request for floor 1.

The FSM detects that the target floor is lower than the current floor.

The system enters the **MOVE** state, and the **Down LED (LEDR[1])** turns ON.

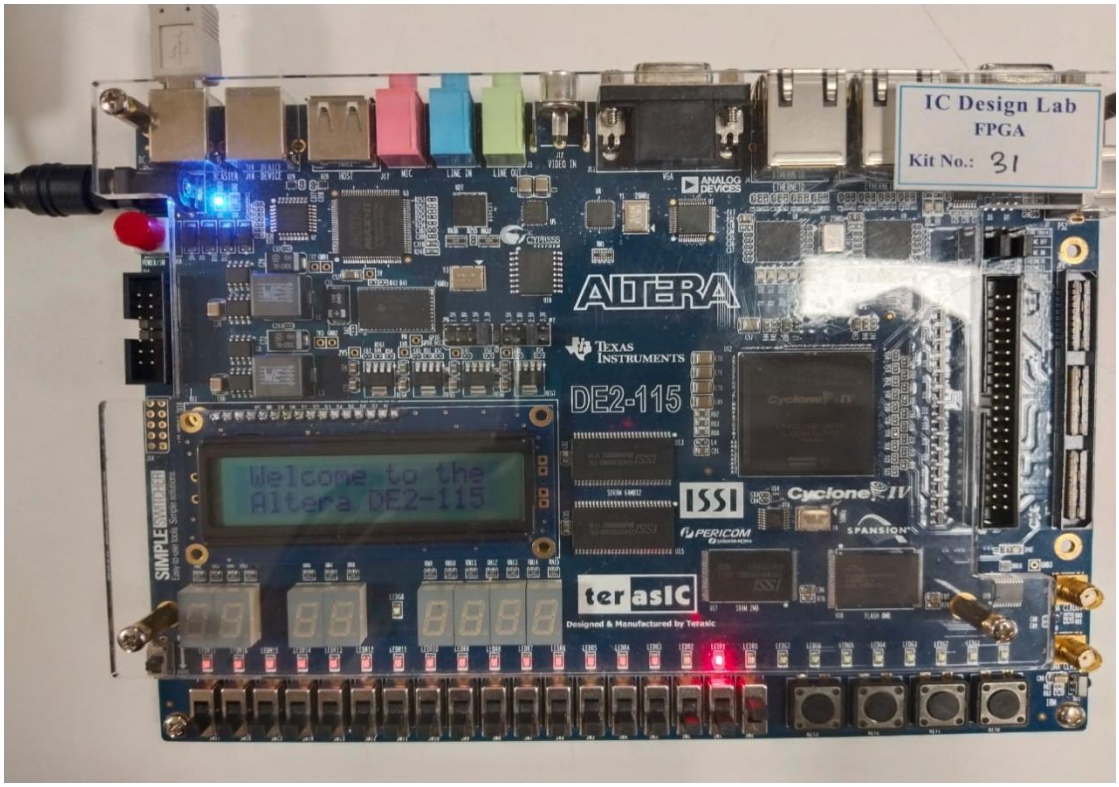
The `current_floor` value decreases in sequence:
→ 3 → 2 → 1

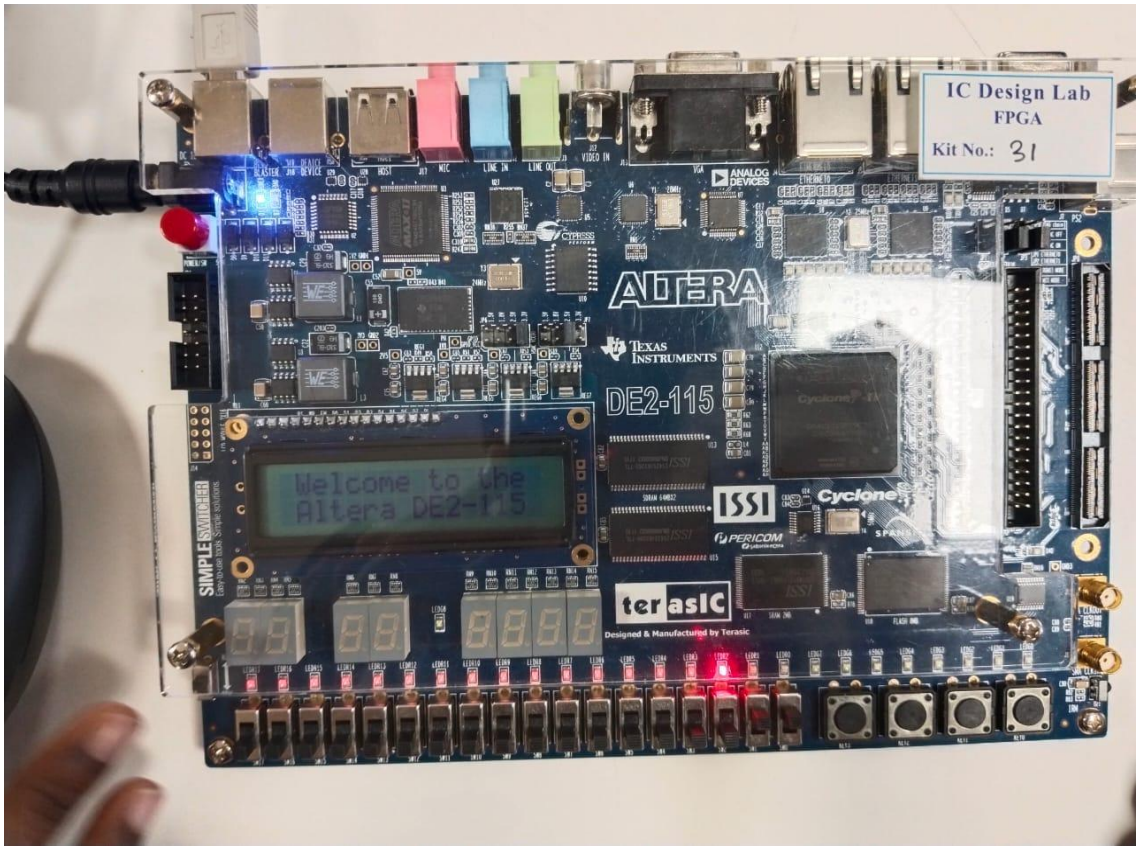
When floor 1 is reached, the FSM enters the **DOOR** state.

The **Door LED (LEDR[2])** glows for a short period, indicating the door is open.

After the timer delay, the FSM resets to **IDLE**, and all movement LEDs turn OFF.

Result:
The elevator accurately moved in the downward direction, stopped at the requested floor, performed the door operation correctly, and returned to idle mode.


IC Design Lab
FPGA
Kit No.: 31
ALTERA
TEXAS INSTRUMENTS
DE2-115
Welcome to the
Altera DE2-115
ISSI
PERICOM
SPANSION
terasIC
Designed & Manufactured by Terasic


IC Design Lab
FPGA
Kit No.: 31
ALTERA
TEXAS INSTRUMENTS
DE2-115
Welcome to the
Altera DE2-115
ISSI
PERICOM
terasIC
Designed & Manufactured by Terasic

6. FUTURE WORK

- **Multi-Floor Expansion**

  Extend the design to handle more floors (e.g., 8 or 16) by increasing floor bits and optimizing the FSM transition logic.

- **Multiple Elevator Coordination**

  Implement communication between two or more elevators to manage requests efficiently in large buildings.

- **Priority Scheduling Algorithm**

  Introduce intelligent scheduling (e.g., shortest-seek-time-first or priority-based request handling) to minimize waiting time.

- **Sensor Integration**

  Add sensors such as proximity or weight sensors to enhance passenger safety and improve system automation.

- **Display and Voice Interface**

  Interface LCD or seven-segment displays to show current floor, and add voice announcements for each floor level.

- **Wireless or IoT Control**

  Enable elevator operation through mobile or IoT-based applications for smart building systems.

- **Power Optimization**

  Apply clock-gating or low-power design techniques to reduce power consumption during idle periods.

7. REFERENCES

[1]N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Addison-Wesley, 2010.
[2] P. K. Meher, "Design and FPGA Implementation of an Elevator Control System Using Finite State Machine," *International Journal of Advanced Research in Electronics and Communication Engineering (IJARECE)*, vol. 6, no. 5, pp. 456–459, May 2017.
[3] R. M. Al-Rawi, "FPGA Implementation of Elevator Controller Using Verilog HDL," *IEEE Conference on Computing, Control, Networking, Electronics and Embedded Systems Engineering (ICCNEEE)*, 2018, pp. 1–5.
[4] Xilinx Inc., *Vivado Design Suite User Guide*, 2021.
[5] Intel Corporation, *Quartus Prime Handbook*, 2020.